

ERP Agentic System

JBM Consulting

Progress Report

Jihed Ben Messaoud

March 2026

Section 1 — Project Overview

1.1 Context and Objectives

The JBM ERP AI Agent is a natural language interface built on top of an enterprise resource planning (ERP) system for JBM Consulting, a fictional Tunisian IT firm. Instead of navigating complex forms and reports, business users type questions in plain language and receive structured, data-backed answers in real time.

The system covers two core ERP modules: Customer Relationship Management (CRM) and Invoicing. A multi-agent architecture, powered by LangChain and a local LLM (qwen2.5:7b via Ollama), routes each user query to the correct specialist agent. Retrieval-Augmented Generation (RAG) enriches every response with semantic context drawn from a 160k-vector ChromaDB store built on top of the data warehouse.

Key innovations include: (1) an LLM-based supervisor agent that routes queries to specialist agents; (2) a write-confirmation safety pattern for all mutating operations; (3) a fully local, privacy-preserving stack (no cloud LLM required); and (4) an Apache Airflow ETL pipeline that ingests, cleans, and loads ~403k raw rows into a star-schema data warehouse.

1.2 Technology Stack

Layer	Technology	Version	Purpose
LLM	qwen2.5:7b via Ollama	local	Natural language understanding and routing
Agent Framework	LangChain	0.2.16	Agent orchestration and tool calling
Vector Store	ChromaDB	0.4.x	RAG embeddings storage
Embedding Model	nomic-embed-text	local	Text vectorization (768 dims)
Backend API	FastAPI	latest	REST endpoints and session handling
Database	PostgreSQL	16	Operational + warehouse data
ORM	SQLAlchemy	2.0	Database access layer
ETL Orchestration	Apache Airflow	2.8.1	Pipeline scheduling and monitoring
Data Processing	pandas	2.0.3	ETL transformations
Fuzzy Matching	rapidfuzz	3.6.1	Company name deduplication
Dashboard	Streamlit	latest	BI visualizations
Charts	Plotly	latest	Interactive charts
Frontend	HTML / CSS / JavaScript	—	Single-page chat interface
Containerization	Docker	—	Airflow deployment

Section 2 — System Architecture

2.1 Overall Architecture Diagram

The system is structured in five horizontal layers: the user-facing frontend (Chat UI and Streamlit dashboards), the FastAPI REST layer, the multi-agent orchestration layer, the data layer (PostgreSQL + ChromaDB + Data Warehouse), and the external Ollama runtime hosting both the LLM and embedding model.

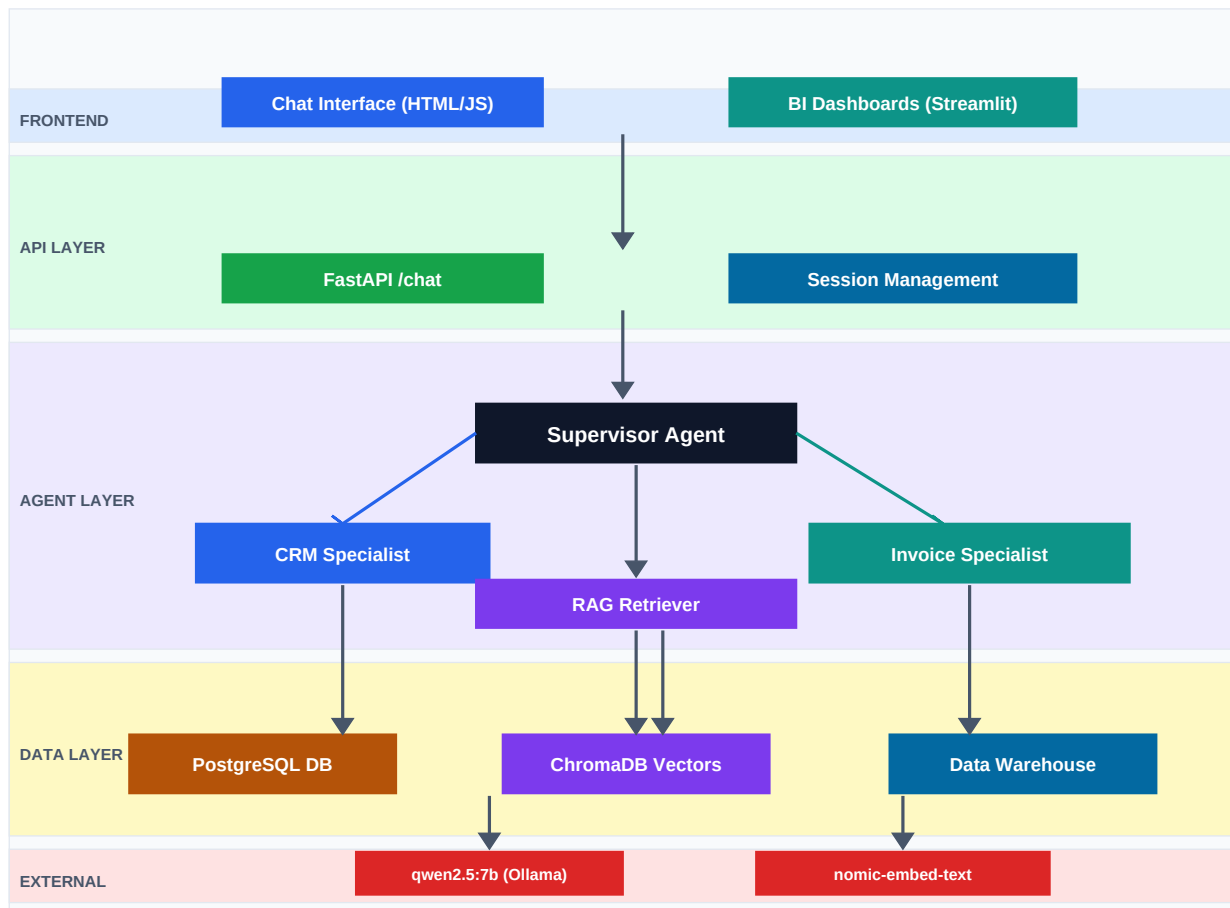


Figure 1 — Five-layer system architecture

2.2 Multi-Agent Routing Diagram

The Supervisor Agent uses LLM-based routing via the supervisor agent to classify each incoming query and dispatch it to the appropriate specialist. The result is routed to the CRM Specialist, the Invoicing Specialist, or handled directly as a general response.

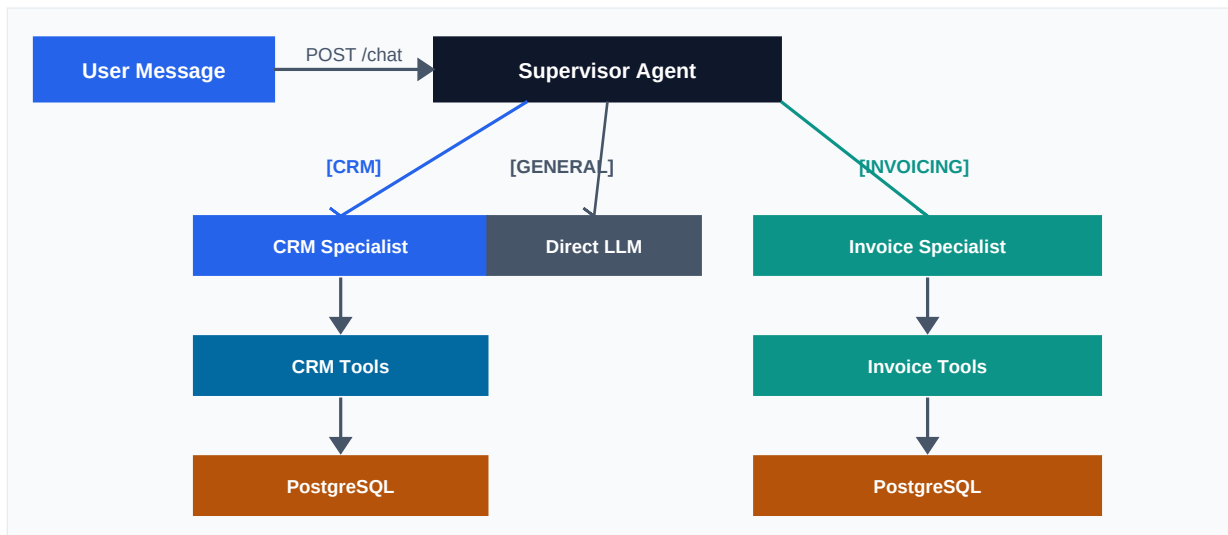


Figure 2 — Multi-agent routing flow

2.3 Request Flow — Step by Step

Every user message triggers a 13-step processing pipeline from the chat interface through to the final response. RAG context is injected at step 6, ensuring that the specialist agent receives semantically relevant warehouse context before tool invocation.

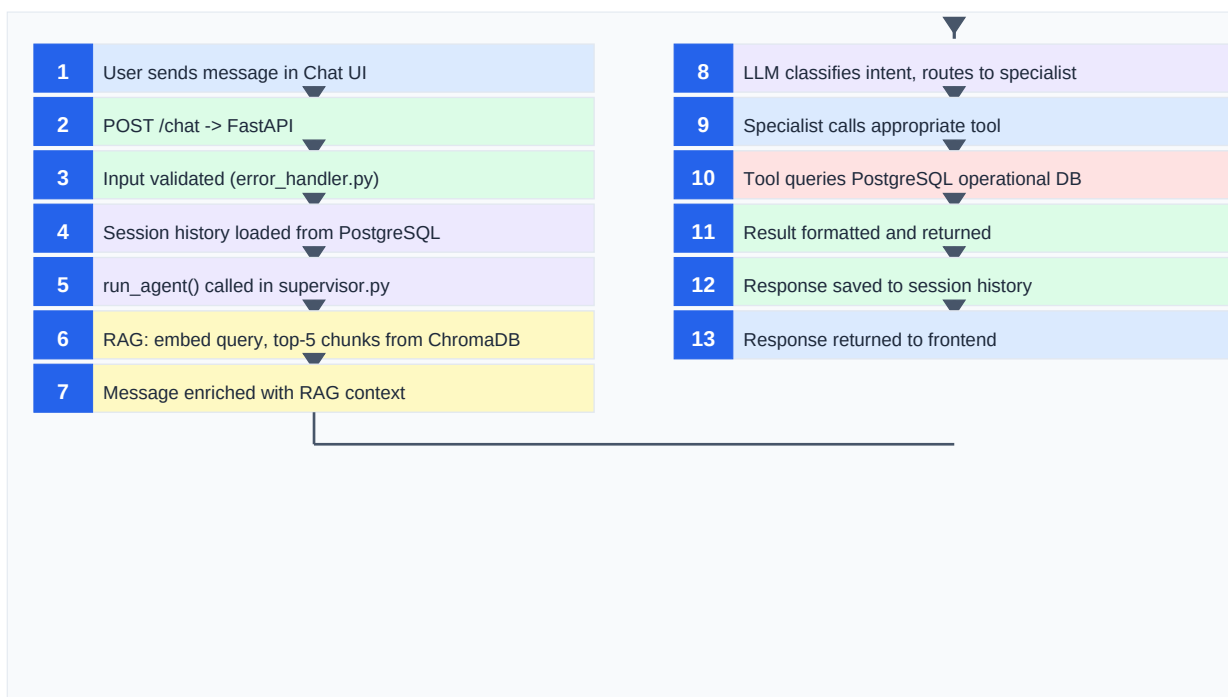


Figure 3 — End-to-end request processing flow

Section 3 — Data Architecture

3.1 Raw Data Sources

File	Rows	Description	Key Issues for ETL
raw_clients.xlsx	98,133	Companies and contacts	Name variations, format noise, duplicates
raw_deals.xlsx	75,944	Sales pipeline history	Value formats, stage variations
raw_activities.xlsx	200,000	CRM activity log	Date formats (12+ variants), outcome variations
raw_invoices.xlsx	29,017	Billing records	Math errors, duplicate invoice numbers
TOTAL	403,094	—	—

3.2 ETL Pipeline Diagram

The ETL pipeline reads from raw Excel files, applies data quality transformations, and loads clean records into the warehouse schema. Key transformations include company name standardization (rapidfuzz, threshold $\geq 85\%$), date normalization across 12+ formats, currency standardization (TND), deal value parsing for mixed formats (e.g., '50k', 'cinquante mille' -> 50000.0 TND), tax recalculation, and duplicate removal.



Figure 4 — ETL pipeline from raw Excel files to warehouse schema

3.3 Data Warehouse Star Schema

The warehouse follows a star schema with three fact tables (fact_deals, fact_revenue, fact_activities) connected to shared dimension tables (dim_client, dim_date). All monetary values are stored in TND.

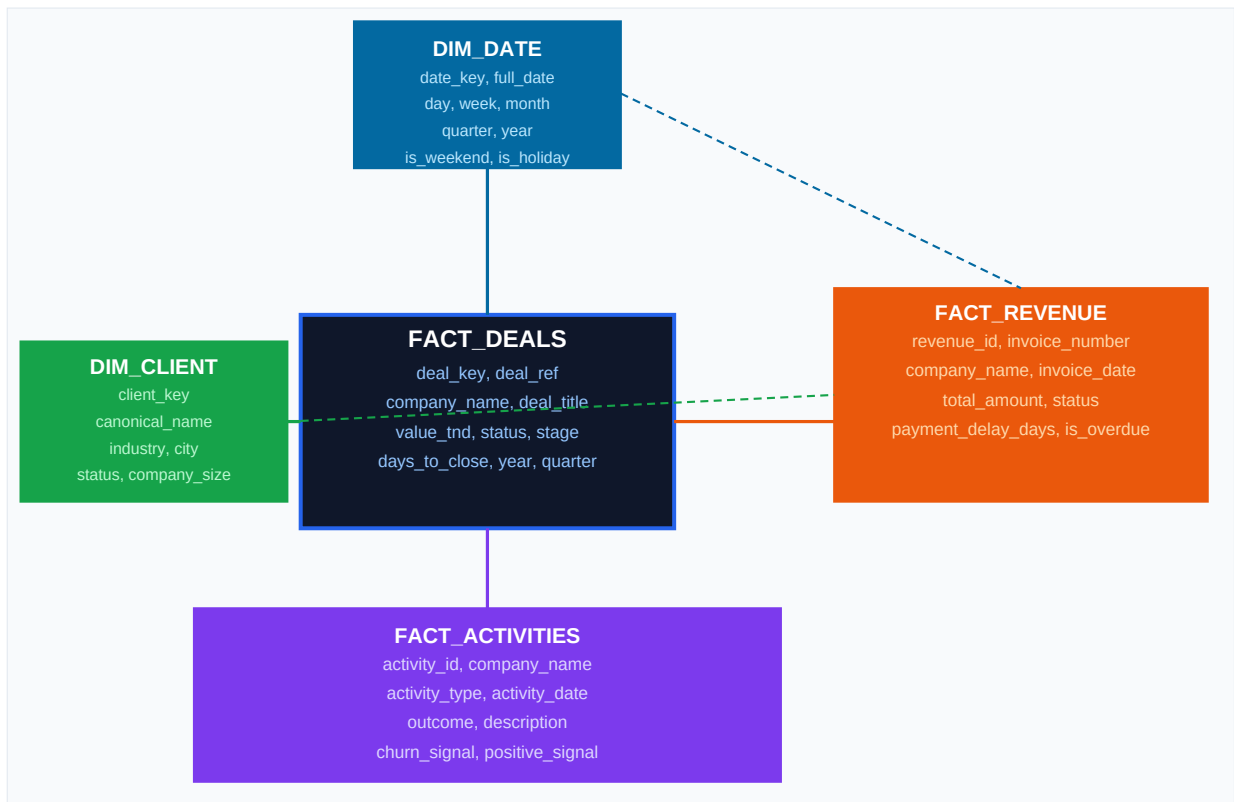


Figure 5 — Star schema: fact and dimension tables

3.4 Warehouse Statistics

Table	Rows	Description
warehouse.dim_client	16,547	Unique deduplicated client/contact records
warehouse.fact_deals	8,000	Cleaned unique deal records
warehouse.fact_activities	200,000	CRM interaction log with churn signals
warehouse.fact_revenue	2,376	Invoice records with payment status
warehouse.dim_date	1,461	Date dimension 2023-2026 incl. Tunisian holidays
TOTAL	227,984	Clean warehouse rows loaded from 403,094 raw records

Section 4 — RAG System

4.1 RAG Architecture

The Retrieval-Augmented Generation system operates in two phases. In the offline embedding phase, warehouse records are chunked (500 chars, 50 overlap), embedded via nomic-embed-text (Ollama, 274 MB, 768 dimensions), and stored in ChromaDB with cosine similarity. In the online retrieval phase, each user query is embedded and the top-5 most similar chunks are prepended to the specialist agent's prompt.

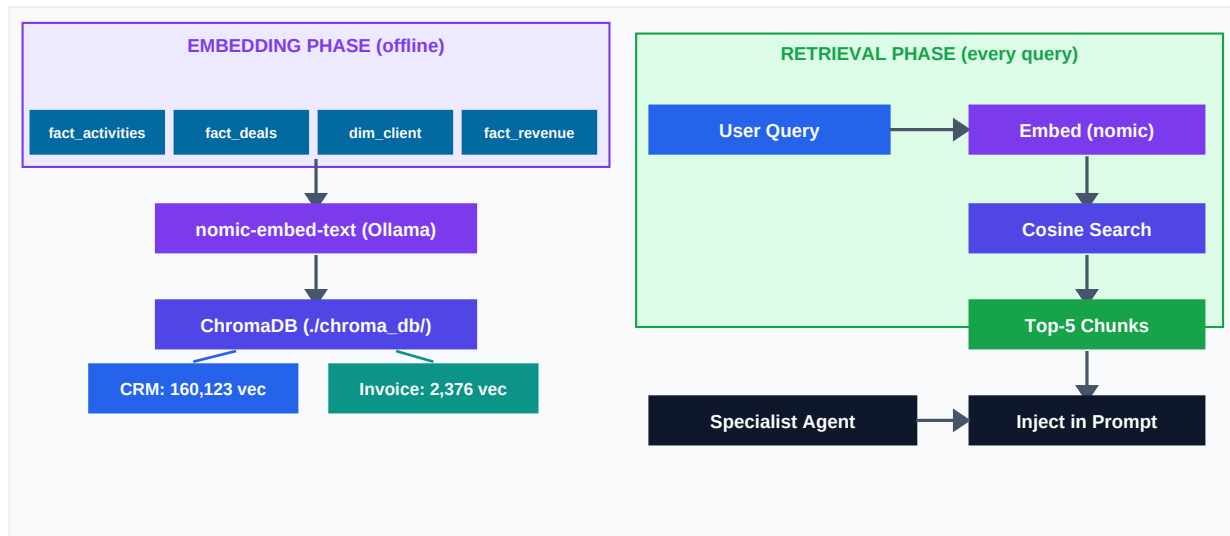


Figure 6 — RAG embedding and retrieval pipeline

4.2 Why RAG is Necessary

Challenge	Without RAG	With RAG
Synonym handling	"workers" != "employees" — missed	Semantic similarity resolves aliases
Client knowledge	Only structured DB fields	Meeting notes, call history retrieved
Churn detection	Not possible from SQL alone	Activity patterns retrieved semantically
Answer depth	Shallow, field-level answers	Evidence-based, context-rich responses
Historical context	Point-in-time DB snapshot only	3-year interaction history surfaced

4.3 Embedding Statistics

Parameter	Value
Total vectors in ChromaDB	162,499
CRM collection (jbm_crm)	160,123 documents
Invoicing collection (jbm_invoicing)	2,376 documents
Embedding model	nomic-embed-text (Ollama, local, 274 MB)

Parameter	Value
Vector dimensions	768
Similarity metric	Cosine
Chunk size	500 characters
Chunk overlap	50 characters
Source tables	fact_activities, fact_deals, dim_client, fact_revenue

Section 5 — Agent System

5.1 CRM Module Tools (12 tools)

Tool Name	Operation	Description
list_companies	READ	List companies with filters (name, city, status)
get_company	READ	Get full company details by name or ID
list_deals	READ	List deals filtered by status or company
get_total_sales	READ	Aggregate sales with date and status filters
get_pipeline_summary	READ	Pipeline statistics by stage
list_contacts	READ	List contacts filtered by company
list_activities	READ	List CRM activities with date/type filters
create_company	WRITE	Create new company (two-phase confirmation)
create_contact	WRITE	Create contact linked to company
create_deal	WRITE	Create new deal opportunity (with confirmation)
update_deal_status	WRITE	Update deal status or stage
create_activity	WRITE	Schedule CRM activity (call, meeting, email)

5.2 Invoicing Module Tools (6 tools)

Tool Name	Operation	Description
list_invoices	READ	List invoices filtered by status or company
get_invoice	READ	Get invoice details with all line items
get_revenue_summary	READ	Revenue aggregation by period in TND
create_invoice	WRITE	Create invoice with line items (confirmation)
mark_invoice_paid	WRITE	Record payment against invoice
send_invoice	WRITE	Mark invoice as sent to client

5.3 Write Operation Safety Flow

All mutating operations (create, update, mark paid) follow a mandatory two-phase confirmation pattern. The first tool call with `confirmed=False` returns a preview. The second call, after the user confirms, executes the write and appends an immutable `audit_log` entry.



Figure 7 — Two-phase write confirmation flow

5.4 Error Handling

Error Type	Where Caught	User-Facing Message
DB Connection Failed	<code>handle_db_error()</code>	Database temporarily unavailable
Tool Execution Failed	<code>handle_tool_error()</code>	Operation failed, please retry
Agent Timeout	<code>handle_agent_error()</code>	Could not complete request in time
Empty Input	<code>validate_user_input()</code>	Please enter a message
Rate Limit	<code>handle_agent_error()</code>	AI service rate limit reached

Section 6 — Business Context — JBM Consulting

6.1 Company Profile

Attribute	Value
Company Name	JBM Consulting (fictional Tunisian IT firm)
Sector	IT Consulting and Solutions
Location	Tunis, Tunisia
Period Covered	January 2023 — March 2026
Services	Data/BI, AI Integration, Cloud Infrastructure
Team Size	8 employees (2023) to 31 employees (2025)
Revenue Growth	~400,000 TND (2023) to ~1,800,000 TND (2025)

6.2 Three-Year Growth Arc

Year	Phase	Team	Win Rate	Key Milestone
2023	Startup	8 employees	28%	Initial client base, small contracts
2024	Growth	18 employees	38%	Landed Tunisie Telecom, enterprise entry
2025	Established	31 employees	50%+	Enterprise portfolio, recurring revenue
2026 Q1	Expansion	31+ employees	~61%	Strong pipeline, forecast positive

6.3 Key Business Metrics from Warehouse

Metric	Value
Total Deals in Warehouse	8,000
Overall Win Rate	49.9%
Won Deals	2,269
Lost Deals	1,411
Active Pipeline — Prospecting	1,613 deals
Active Pipeline — Proposal	922 deals
Active Pipeline — Negotiation	554 deals
Total Invoices	2,376
Paid Invoices	1,845 (77.6%)
Pending Invoices	386 (16.2%)

Metric	Value
Overdue Invoices	29 (1.2%)
Cancelled Invoices	116 (4.9%)
Total CRM Activities	200,000 interactions over 3 years
Churn/Positive Signals	Encoded in 160,123 activity embedding vectors

Section 7 — Project Management — Scrum

7.1 Scrum Overview

The project follows an adapted Scrum methodology, with one-week sprints and a solo development team. The product backlog is maintained collaboratively with the company supervisor. Ceremonies are lightweight: sprint reviews conducted periodically with supervisors.

Role	Person / Entity
Product Owner	Company supervisor — Eita Consulting
Scrum Master	Jihed Ben Messaoud
Development Team	Jihed Ben Messaoud (solo developer)
Sprint Duration	1 week

7.2 Sprint Plan

Sprint	Goal	Deliverables	Status
Sprint 1	Project setup + Architecture	Environment, DB models, seed data	Done
Sprint 2	Agent foundation	Tools, MCP layer, basic agents	Done
Sprint 3	Multi-agent system	Supervisor, specialist agents, routing	Done
Sprint 4	Data pipeline	Raw data generation, ETL, warehouse	Done
Sprint 5	RAG system	ChromaDB, embeddings, retrieval layer	Done
Sprint 6	BI Dashboards	Streamlit dashboards, KPI charts	In Progress
Sprint 7	Evaluation + Security	Test framework, JWT authentication, RBAC	Planned
Sprint 8	UI + Deployment	New frontend, Docker containerization	Planned
Sprint 9	Report + Defense	Final documentation, diagrams, presentation	Planned

7.3 Product Backlog

Priority	Item	Sprint	Status
High	Evaluation framework (60+ test queries, accuracy metrics)	7	Planned
High	Agent performance tuning based on evaluation results	7	Planned
Medium	JWT authentication + role-based access control	7	Planned
Medium	New production-quality SaaS frontend UI	8	Planned
Medium	Docker containerization and deployment	8	Planned

Priority	Item	Sprint	Status
Low	Churn prediction model (ML on activity signals)	9	Planned
Low	Revenue forecasting model	9	Planned
Low	Additional ERP module (HR or Projects if time permits)	9	If time permits

Section 8 — Project Status

8.1 Completed Components

Component	Status	Details
ERP Modules (CRM + Invoicing)	Complete	Full CRUD via natural language, 18 tools total
Multi-Agent Architecture	Complete	Supervisor + 2 specialist agents, two-tier routing
RAG System	Complete	162,499 vectors in ChromaDB, cosine similarity
Raw Data Generation	Complete	403,094 rows across 4 Excel files
ETL Pipeline	Complete	58-second runtime, all validation checks passed
Data Warehouse	Complete	5 tables, 227,984 clean rows, star schema
Error Handling	Complete	14 error types, centralized ERPErrorType enum
Chat Frontend	Complete	Dark SaaS UI, single-page HTML/CSS/JS
BI Dashboards	In Progress	Streamlit, 2 dashboards under active development

8.2 Planned Components

Component	Priority	Description
Evaluation Framework	High	60+ test queries, accuracy and relevance metrics
Agent Performance Tuning	High	Improvements based on evaluation results
JWT Authentication + RBAC	Medium	Role-based access: CEO, Sales Manager, Accountant
New SaaS UI	Medium	Production-quality single-page frontend
Docker Deployment	Medium	Full containerization of API + dependencies
Churn Prediction Model	Low	ML model on activity churn signals
Revenue Forecasting	Low	Time-series forecasting on fact_revenue
Additional ERP Module	Low	HR or Projects module if time permits
Final Report	Final	Scrum methodology, full documentation

This document represents the technical state of the JBM ERP AI Agent project as of Mars 2026. All monetary values are expressed in TND (Tunisian Dinar). The system is built entirely on open-source components with no dependency on commercial cloud LLM APIs.